0207. Course Schedule

Medium | Category: Graphs | LeetCode: https://leetcode.com/problems/course-s

Problem

- NumCourses and prerequisite pairs [a, b] (take b before a). Can all courses be finished? Equivalent to: is the directed graph acyclic?

Intuition

- Kahn's algorithm: repeatedly take a course with no remaining prerequisites (in-degree 0) and remove its outgoing edges.
- If we manage to take all courses, there is no cycle.

Brute Force

- DFS with three colours (unvisited / visiting / done) detects a back edge; equally optimal.

Optimal Approach

- Build adjacency b → a and indegree[a]++.
- Queue all courses with indegree 0.
- Pop, count++, decrement neighbours' indegree, enqueue those reaching 0.
- Return count == numCourses.

Worked Example

```
n = 4, prereqs [[1,0],[2,1],[3,2],[1,3]].

indegree: 0:0 1:2 2:1 3:1
queue [0] -> take 0 -> indegree 1 becomes 1
queue empty, taken 1 of 4 -> cycle 1->2->3->1 -> false
```

Java Solution

```java
public class CourseSchedule {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        int[] indeg = new int[numCourses];
        for (int[] p : prerequisites) { adj.get(p[1]).add(p[0]); indeg[p[0]]++; }
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.add(i);
        int taken = 0;
        while (!q.isEmpty()) {
            int c = q.poll();
            taken++;
            for (int nx : adj.get(c)) if (--indeg[nx] == 0) q.add(nx);
        }
        return taken == numCourses;
    }
}
```

Complexity

- Time: O(V + E). Space: O(V + E).

Pitfalls

- Edge direction: [a, b] means b → a.